

ESPOL
(logo placeholder)

Escuela Superior Politecnica del Litoral
Facultad de Ingenieria en Electricidad y Computacion

Software Engineering I

Leodega

Storage Marketplace

Software Requirements Specification

Team Members

Darwin Leonardo Díaz González
Geovanny Jahir Diaz Loor
Carla Alejandra Gutiérrez Córdova
Johao Carlos Dorado Quinde

Professor

Mgti. Vanessa Jurado Alexandra Vite

June 2026

PAO II – 2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Problem Definition

The storage of goods is a recurring logistical need for both individuals and companies, yet the available supply of storage space is fragmented and largely underutilised. Many property owners hold unused spaces — warehouses, garages, rooms, containers, basements, and attics — that could be offered to third parties, while many other individuals and organisations struggle to find flexible, adequately sized, and conveniently located storage alternatives. At present, the matching between supply and demand of storage space is mostly carried out through informal, unmediated channels, which makes the process slow, opaque, and risky for both parties.

This situation gives rise to several concrete problems that the present software project intends to solve:

- **Lack of trust and verifiable information.** Prospective renters cannot easily verify the conditions, dimensions, location, or legal compliance of a storage space before committing to a rental, and owners lack a structured reputation mechanism to demonstrate reliability.
- **Absence of formal moderation and compliance control.** There is no centralised authority that validates listings — for example, that the mandatory fire department permit is in order — before they are exposed to the public.
- **Inefficient reservation and contracting.** The coordination of availability windows, the detection of date conflicts, the formalisation of payment, and the activation of contracts are performed manually, which is error-prone and does not scale.
- **Limited communication and incident management.** Once a rental begins, the parties lack a dedicated channel to coordinate access, resolve incidents, or escalate problems to a moderator, and there is no structured report-and-resolution workflow.
- **No support for organisational (company-level) rentals.** Companies that need storage space for their collaborators cannot manage rentals under a collective identity, invite members, or audit the actions performed in the organisation’s name.
- **No mobile access.** The management of published spaces and the exploration of available spaces are confined to a web experience, which limits on-the-go use by owners and clients.
- **No integration with external inventory systems.** Clients who already manage their stock in an external system must re-enter their product data manually, with no mechanism

to query or import inventory automatically.

To address these problems, the product owner supplied an existing web platform, **Leodega**, as the starting point of the project. This platform already provides a collaborative marketplace for storage space — including listing publication, search, reservations, ratings, reports, moderation, notifications, and messaging — but it must be extended with the organisational, mobile, integration, compliance, and control capabilities specified in the user stories, which constitute the primary source of functional requirements for this document.

1.2 Project Context

The product owner provided the team with an existing full-stack web application, **Leodega**, as the start state of the project. Leodega is an online marketplace that connects *landlords* (storage owners, hereafter *Storage Managers*) with *tenants* (renters, hereafter *Clients*), and is overseen by a *System Administrator*. At the moment of delivery, the platform was in the integration and testing phase and had been developed under the Scrum methodology.

The inherited platform is implemented with the following technology base, which constitutes the foundation on which the requirements specified in this document are built:

- **Backend.** Laravel 12 (PHP 8.2+) with Laravel Sanctum token-based authentication, Eloquent ORM, and a PostgreSQL database in production (SQLite in local development), containerised with Docker Compose and deployable to Railway.
- **Frontend.** React 19 with TypeScript and Vite, React Router DOM with protected and role-based route guards, Tailwind CSS, Axios with a Bearer-token interceptor, Leaflet/React-Leaflet maps, and i18next internationalisation; deployable to Vercel.
- **Existing capabilities.** User authentication and registration, role-based access control (administrator, landlord, tenant), publication and moderation of storage spaces, search and detail views, reservations with conflict detection, ratings, reports, in-app notifications, conversations and messages, a payments registry, and role-based dashboards.

The team's task is to evolve this inherited platform so that it satisfies the functional requirements expressed in the user stories, extending it with organisational accounts, a mobile companion named *Leodeguita*, external inventory integration, Excel inventory import, fire-department-permit compliance, an administrative control dashboard, and account moderation. The system operates in a web environment and requires an Internet connection; the Leodeguita companion extends the principal functions to mobile devices.

1.3 General Objective

To evolve the Leodega platform into a complete, reliable, and mobile-accessible storage marketplace that connects Storage Managers and Clients — individually or under organisations — through a moderated and trustworthy system that integrates reservation, payment, communication, incident-reporting, and inventory-integration capabilities, as specified in the user stories.

1.4 Specific Objectives

The following specific objectives support the general objective and correspond to the functional-requirement groups defined in the user stories:

1. **Storage management.** Enable Storage Managers to register, edit, and delete warehouses — including the mandatory fire department permit — and to view, cancel, and communicate about their reservations. (References: HUG-01 to HUG-08.)
2. **Client experience.** Enable Clients to search and filter warehouses, view their details, reserve and pay for them, manage their reservations, communicate with managers, report incidents, rate finished rentals, and integrate their external inventory systems or import inventory from Excel files. (References: HUC-01 to HUC-14.)
3. **Platform administration.** Enable Administrators to validate fire department permits and moderate listings, block and reactivate user accounts, monitor a control dashboard with key metrics, and review and resolve reports. (References: HUA-01 to HUA-04.)
4. **Organisational accounts.** Enable the creation of organisations, the invitation and acceptance of client members, the switching between personal and organisation contexts, business-level reservations, and the administration and auditing of organisational activity. (References: HUE-01 to HUE-06.)
5. **Mobile companion (Leodeguita).** Provide mobile access for authentication with Leodega credentials, company selection, warehouse publication, and browsing of available warehouses. (References: HUL-01 to HUL-06.)

1.5 Project Scope

The scope of the system is defined by the functional requirements derived from the user stories, which are summarised in Chapter 2 and detailed in Appendix A. The inherited Leodega platform provides the technological and functional foundation described in Section 1.2; the present specification governs the capabilities that the delivered system must satisfy.

Functional scope by user level

Storage Manager (landlord). The system allows the Storage Manager to: receive and reply to pre-booking inquiries; receive payment-confirmation notifications and view active contracts; chat with clients during active rentals; register a warehouse with dimensions, location, photos, monthly rate, and fire department permit; view reservations with filtering and detail; cancel future paid reservations with a mandatory reason; delete warehouses that have no active or future reservations; and edit warehouse information. (HUG-01 to HUG-08.)

Client (tenant). The system allows the Client to: search and filter warehouses by location, size, and price, with a map view; view warehouse details; reserve available warehouses with date-conflict validation; complete a simulated payment to confirm a reservation; communicate with managers before and during rentals; view and manage active and historical reservations; cancel future confirmed reservations; report incidents; rate finished rentals; accept or reject organisation

invitations; register an external inventory endpoint and its field mapping; view synchronised inventory; and import inventory from an Excel file. (HUC-01 to HUC-14.)

Administrator. The system allows the Administrator to: validate the fire department permit and approve or reject warehouse publications; block and reactivate user accounts with audit logging; monitor a control dashboard with warehouse, user, transaction, reservation, and unresolved-report metrics; and review, reply to, and resolve reports. (HUA-01 to HUA-04.)

Organisation Administrator and organisation members. The system allows an Organisation Administrator to: create an organisation; invite clients as members; access a business administrative panel with members, reservations, and history; and delete business reservations with a mandatory reason. It allows member clients to: switch between personal and organisation contexts; reserve on behalf of the organisation; and share visibility of business reservations, while being prevented from deleting business reservations. (HUE-01 to HUE-06.)

Leodeguita users (all roles). The mobile companion allows any Leodeguita user to authenticate with existing credentials; allows a client to select the operating company; allows a Storage Manager to publish a warehouse for review and to view their published warehouses; and allows a client to browse and view the detail of available warehouses. (HUL-01 to HUL-06.)

Out of scope

The following items are explicitly out of scope of this specification:

- **Real financial payment processing.** The payment flow is simulated (HUC-05); integration with real payment gateways or banking processors is not required.
- **ERP integration and electronic invoicing.** These are part of the project vision but are not yet implemented and are not covered by any user story; they are excluded from this specification.
- **A general-purpose React Native mobile application.** A full mobile application is planned for future versions; only the Leodeguita companion flows specified in HUL-01 to HUL-06 are in scope.
- **Inherited-baseline features not re-specified in the user stories** (for example, favourites, session-token management, and password reset) are retained from the inherited platform as-is and are not re-specified as functional requirements in this document.

Chapter 2

Requirements Summary

This chapter consolidates the functional and non-functional requirements of the Leodega platform in summary form. The functional requirements are expressed as User Stories and are detailed in Appendix A; the non-functional requirements are detailed in Appendix B. The summaries presented here provide a quick, traceable overview and do not replace the detailed specifications given in the appendices.

2.1 Functional Requirements

The functional requirements of the system are derived exclusively from the User Stories supplied in `user_stories.md`, which constitute the primary source of functional requirements per the editorial instructions. They are organised by the five roles that interact with the system: **Storage Manager** (HUG), **Client** (HUC), **Administrator** (HUA), **Organisation / Company** (HUE), and the **Leodeguita** mobile companion (HUL). Table ?? lists every requirement by ID, a concise descriptive name, and the reference to its detailed User Story in Appendix A.

Table 2.1: Functional Requirements Summary

ID	Requirement Name	Reference
<i>Storage Manager (HUG)</i>		
HUG-01	Pre-booking inquiry reception and reply	App. A, HUG-01
HUG-02	Payment-confirmation notification and contract activation	App. A, HUG-02
HUG-03	Active-contract chat with clients	App. A, HUG-03
HUG-04	Warehouse registration with fire department permit	App. A, HUG-04
HUG-05	Reservation list viewing, filtering, and detail	App. A, HUG-05
HUG-06	Cancellation of paid future reservations	App. A, HUG-06
HUG-07	Warehouse deletion (no active or future reservations)	App. A, HUG-07
HUG-08	Warehouse information editing	App. A, HUG-08
<i>Client (HUC)</i>		

continued on next page

Table ?? (continued)

ID	Requirement Name	Reference
HUC-01	Warehouse search, filtering, and map view	App. A, HUC-01
HUC-02	Warehouse detail view	App. A, HUC-02
HUC-03	Warehouse reservation with date-conflict detection	App. A, HUC-03
HUC-04	Pre-booking message to manager	App. A, HUC-04
HUC-05	Simulated payment confirmation	App. A, HUC-05
HUC-06	Active-reservation chat	App. A, HUC-06
HUC-07	Reservation history management and cancellation	App. A, HUC-07
HUC-08	Incident reporting and tracking	App. A, HUC-08
HUC-09	Warehouse and manager rating and review	App. A, HUC-09
HUC-10	External inventory endpoint registration	App. A, HUC-10
HUC-11	External inventory field mapping	App. A, HUC-11
HUC-12	Synchronised inventory viewing	App. A, HUC-12
HUC-13	Excel inventory import	App. A, HUC-13
HUC-14	Organisation invitation acceptance or rejection	App. A, HUC-14
<i>Administrator (HUA)</i>		
HUA-01	Fire department permit validation	App. A, HUA-01
HUA-02	Administrative control dashboard	App. A, HUA-02
HUA-03	User account blocking and reactivation	App. A, HUA-03
HUA-04	Report review and resolution	App. A, HUA-04
<i>Organisation / Company (HUE)</i>		
HUE-01	Organisation member invitation	App. A, HUE-01
HUE-02	Personal / organisation context switching	App. A, HUE-02
HUE-03	Business administrative panel	App. A, HUE-03
HUE-04	Organisation creation	App. A, HUE-04
HUE-05	Business-level reservation	App. A, HUE-05
HUE-06	Business reservation deletion control	App. A, HUE-06
<i>Leodeguita mobile companion (HUL)</i>		
HUL-01	Leodeguita login with Leodega credentials	App. A, HUL-01
HUL-02	Company selection in Leodeguita	App. A, HUL-02
HUL-03	Mobile warehouse publication for review	App. A, HUL-03
HUL-04	Published warehouses list (mobile)	App. A, HUL-04
HUL-05	Available spaces browsing (mobile)	App. A, HUL-05
HUL-06	Warehouse detail view (mobile)	App. A, HUL-06

In total, the specification comprises 38 functional requirements: 8 for the Storage Manager, 14 for the Client, 4 for the Administrator, 6 for the Organisation, and 6 for the Leodeguita mobile companion. Each entry in Table ?? is traceable to a User Story in Appendix A, where the full statement, priority, estimate, sprint, and acceptance criteria are provided.

2.2 Non-Functional Requirements

The non-functional requirements of the system are sourced from `non_functional_requirements.md` and are categorised according to Ian Sommerville’s classification, which distinguishes three top-level families of quality requirements:

- **Product requirements** constrain the behaviour of the software itself (e.g., efficiency, reliability, usability, security, portability).
- **Organisational requirements** arise from the policies, procedures, and development standards of the team and institution (e.g., implementation and process standards).
- **External requirements** arise from factors outside the system and its development, such as regulatory or legislative obligations.

Table ?? lists each non-functional requirement by ID, a concise name, its Sommerville category, and the reference to its detailed specification in Appendix B, where the validation criteria are provided.

Table 2.2: Non-Functional Requirements Summary (Sommerville classification)

ID	Requirement Name	Sommerville Category	Reference
RNF-01	Performance (response time)	Product — Efficiency	App. B, RNF-01
RNF-02	Availability	Product — Reliability	App. B, RNF-02
RNF-03	Security: Authentication	Product — Security	App. B, RNF-03
RNF-04	Security: Data protection	Product — Security	App. B, RNF-04
RNF-05	Usability	Product — Usability	App. B, RNF-05
RNF-06	Compatibility (browsers and devices)	Product — Portability	App. B, RNF-06
RNF-07	Internationalization	Product — Portability	App. B, RNF-07
RNF-08	Maintainability	Organisational — Implementation	App. B, RNF-08
RNF-09	Testability	Organisational — Implementation	App. B, RNF-09
RNF-10	Scalability	Product — Efficiency	App. B, RNF-10
RNF-11	Auditability	External — Regulatory	App. B, RNF-11

The categorisation in Table ?? follows Sommerville’s framework: eight requirements (RNF-01, RNF-02, RNF-03, RNF-04, RNF-05, RNF-06, RNF-07, and RNF-10) constrain properties of the product; two requirements (RNF-08 and RNF-09) constrain the development process and its implementation standards; and one requirement (RNF-11) responds to external regulatory

accountability needs through audit-trail traceability. Every non-functional requirement is stated in testable terms and is accompanied by quantitative validation criteria in Appendix B, in compliance with the editorial instruction that every requirement be testable and that validation criteria be included for every non-functional requirement.

Chapter 3

Project Planning and Control

To plan and control the project, we first organized all the gathered requirements (user stories) as individual tasks and defined their dependencies, prioritization and sequencing across the five development sprints. After establishing all requirements and their dependencies, we developed a Program Evaluation and Review Technique (PERT) diagram, also known as an Activity-on-Arrow (AOA) diagram. This diagram, shown in Figure ??, helps us schedule the project activities by determining the earliest start times and required completion times for each event. The diagram reveals a total project duration of **52 time units (days)**, with the critical path highlighted in red. The critical path runs through the activities HST-001 → HST-002 → HST-007 → HST-009 → HST-011 → HST-013 → HST-014 → HST-034 → HST-035 → HST-036; that is, the chain that runs from authentication and warehouse registration, through administrator moderation, catalog search, warehouse detail, reservation and simulated payment, and ends with the inventory-integration module (external connection, field mapping and inventory view), which concentrates the heaviest tasks of the project (8, 8 and 5 days).

The project is structured into five sprints, each focusing on a specific aspect of the platform’s development:

- **Sprint 1: Authentication and Warehouse Management Foundations**

- HST-001: Shared login for the web and the Leodeguita mobile app.
- HST-002: Warehouse registration including the fire-department permit.
- HST-003: Warehouse publishing from the mobile app.
- HST-004: Editing a published warehouse.
- HST-005: Logical deletion of a warehouse.
- HST-006: “My warehouses” listing (mobile).
- HST-007: Administrator moderation and approval of warehouses.
- HST-008: Blocking and reactivating user accounts.

- **Sprint 2: Catalog, Reservations and Payment**

- HST-009: Search and filtering of approved warehouses.
- HST-010: Unified space listing (mobile).
- HST-011: Warehouse detail view.

- HST-012: Warehouse detail view (mobile).
- HST-013: Period reservation with temporary availability hold.
- HST-014: Simulated instant-book payment.
- HST-015: Payment-confirmation notification for the manager.
- HST-016: Manager reservation list.
- HST-017: Reservation cancellation by the manager.
- **Sprint 3: Organizations (Multi-company)**
 - HST-018: Organization creation from a client account.
 - HST-019: Personal/organization context switcher.
 - HST-020: Organization selector (mobile).
 - HST-021: Inviting client users to the organization.
 - HST-022: Accepting or rejecting invitations.
 - HST-023: Organization administration panel.
 - HST-024: Reserving on behalf of an organization.
 - HST-025: Protection of organization reservations.
- **Sprint 4: Messaging, Reports and Reviews**
 - HST-026: Pre-booking inquiry (client side).
 - HST-027: Pre-booking inquiry (manager side).
 - HST-028: Reservation chat (client side).
 - HST-029: Reservation chat (manager side).
 - HST-030: Client reservation management.
 - HST-031: Problem reporting.
 - HST-032: Report management (administrator).
 - HST-033: Rating and reviews.
- **Sprint 5: Inventory Integration and Metrics**
 - HST-034: External inventory connection (pull model).
 - HST-035: Field mapping between the external system and Leodega.
 - HST-036: On-demand inventory view.
 - HST-037: Inventory import from an Excel file.
 - HST-038: Administrator metrics dashboard.

Based on our PERT analysis, we created the comprehensive schedule shown in Table ???. This schedule details each requirement’s sprint assignment, predecessors, duration and resource allocation (staff), along with the earliest event time (EET, i.e. the earliest start), the total float and the latest event time (LET, i.e. the latest finish), all expressed in days. Activities whose float is zero belong to the critical path and are marked with a bullet (•). Note that HST-002 and HST-003 run in parallel and both have zero float; the diagram draws the red critical path through HST-002.

AOA_Leodega_circulos.png

Figure 3.1: Activity-on-Arrow (PERT) diagram for Leodega. The critical path is shown in red and the total project duration is 52 days.

Table 3.1: Project schedule for Leodega: sprint, dependencies, timing, resource allocation and CPM analysis (EET, float, LET).

ID	Sprint	Pred.	Dur.	Staff	EET	Float	LET	CP	
HST-001	1	—		3	1	0	0	3	•
HST-002	1	001		5	2	3	0	8	•
HST-003	1	001		5	2	3	0	8	•
HST-004	1	002		5	2	8	39	52	
HST-005	1	002		3	1	8	41	52	

continued on next page

Table 3.1 – *continued from previous page*

ID	Sprint	Pred.	Dur.	Staff	EET	Float	LET	CP
HST-006	1	002, 003	2	1	8	42	52	
HST-007	1	002, 003	5	2	8	0	13	•
HST-008	1	001	5	2	3	36	44	
HST-009	2	007	5	2	13	0	18	•
HST-010	2	007	3	1	13	34	50	
HST-011	2	009	3	1	18	0	21	•
HST-012	2	010	2	1	16	34	52	
HST-013	2	011	5	2	21	0	26	•
HST-014	2	013	5	2	26	0	31	•
HST-015	2	014	3	1	31	18	52	
HST-016	2	014	3	1	31	15	49	
HST-017	2	016	3	1	34	15	52	
HST-018	3	001	5	2	3	28	36	
HST-019	3	018	5	2	8	31	44	
HST-020	3	018	3	1	8	41	52	
HST-021	3	018	5	2	8	28	41	
HST-022	3	021	3	1	13	28	44	
HST-023	3	019, 022	5	2	16	23	49	
HST-024	3	019, 014	5	2	31	13	49	
HST-025	3	023, 024	3	1	36	13	52	
HST-026	4	011	3	1	21	23	47	
HST-027	4	026	5	2	24	23	52	
HST-028	4	014	5	2	31	11	47	
HST-029	4	028	5	2	36	11	52	
HST-030	4	014	3	1	31	18	52	
HST-031	4	014	3	1	31	15	49	
HST-032	4	031	3	1	34	15	52	
HST-033	4	014	3	1	31	18	52	
HST-034	5	014	8	3	31	0	39	•
HST-035	5	034	8	3	39	0	47	•
HST-036	5	035	5	2	47	0	52	•
HST-037	5	014	5	2	31	16	52	
HST-038	5	007, 008, 014	8	3	31	13	52	

Because the critical-path duration of 52 days assumes unlimited resources, we then performed

resource-limited scheduling subject to a cap of four team members. We first built a non-optimized schedule, shown in Figure ??, which completes the project in **87 days**. We then optimized it by levelling resource usage and reordering non-critical activities within their available float; the resulting optimized schedule, shown in Figure ??, completes the project in **85 days**. This optimization saves two days while never exceeding the four-person limit and never delaying any critical activity.

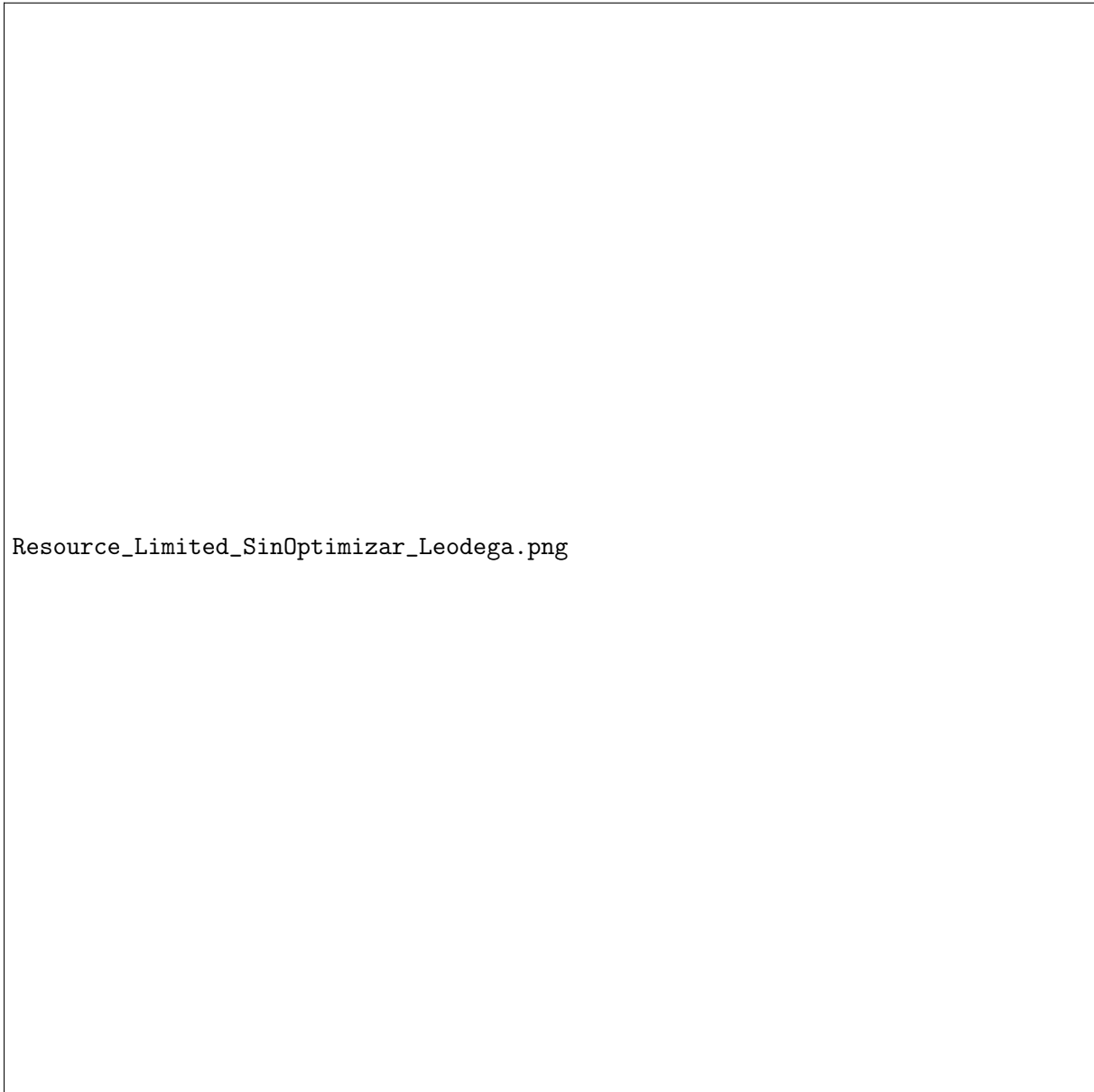


Figure 3.2: Non-optimized resource-limited scheduling chart (cap = 4 people, total duration = 87 days).

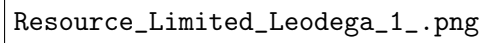
The image is a placeholder for a resource-limited scheduling chart. It is a rectangular area with a thin black border. The text "Resource_Limited_Leodega_1_.png" is centered within this area.

Figure 3.3: Optimized resource-limited scheduling chart (cap = 4 people, total duration = 85 days).

Chapter 4

System Prototype

The Leodega prototype is a high-fidelity web application developed using React and Tailwind CSS. The system supports three primary user roles: **Tenant**, **Landlord/Manager**, and **Administrator**, each with distinct interaction flows. The following sections document the main screens of the prototype, organised by user role.

4.1 General Screens

The following screens are shared across all user types, covering the public landing page, authentication flow, and user profile selection.

Figure 4.1: Landing Page: hero section with value proposition and mobile platform access links.

Figure 4.2: Landing Page: “How We Work” section presenting the warehouse rental process in three steps.

Figure 4.3: Landing Page: “Why Choose Us” section highlighting the platform’s key differentiators.

Figure 4.4: Landing Page: “Popular Warehouses” section showing featured listings with price and rating.

4.1.1 Authentication

The authentication flow allows users to create a new account or sign in using their own credentials or through third-party providers (Facebook, Google, Apple).

Figure 4.5: Login screen with email and password form and social login options.

Figure 4.6: Registration screen: account creation form with personal data and password confirmation.

Figure 4.7: User profile selection: Tenant, Landlord/Manager, or Administrator.

4.2 Tenant

The Tenant flow enables users to search and filter available warehouses, view detailed information, complete the booking and payment process, and manage their active rentals.

4.2.1 Warehouse Catalog

The catalog displays available warehouses with filters for city, size, and price. Each card shows the type, location, surface area, rating, and monthly price.

Figure 4.8: Warehouse Catalog: grid view with city, size, and price filters (15 listings shown).

4.2.2 Warehouse Detail

The detail view presents a photo gallery, full description, features, assigned manager, availability calendar, location map, and cost breakdown.

Figure 4.9: Warehouse Detail: photo gallery, features, and cost summary (Step 1 of the booking flow).

Figure 4.10: Warehouse Detail: manager information, availability calendar, and location map.

4.2.3 Booking and Payment Process

The booking flow consists of three steps: Detail, Payment, and Receipt. The payment step collects card information and shows the final cost breakdown before confirmation.

Figure 4.11: Step 2 – Payment: card details form and total cost summary (\$3,010 USD).

4.2.4 My Bookings

The bookings panel groups rentals into Active, Completed, and Cancelled tabs, with options to cancel or report an issue.

Figure 4.12: My Bookings: active rentals list with period, amount, and booking reference code.

4.2.5 Messages

The messaging module allows tenants to communicate with warehouse managers for pre-booking inquiries and during active rental periods.

Figure 4.13: Tenant Messages: inbox with per-warehouse conversations and real-time chat with the manager.

4.2.6 Inventory

The inventory module allows tenants to connect an external system via a read-only endpoint URL to display stock data directly within Leodega.

Figure 4.14: Inventory: external system connection via endpoint URL and read-only access token.

4.3 Landlord / Manager

The Landlord/Manager role provides a management panel for publishing warehouses, tracking incoming bookings, communicating with tenants, and monitoring the occupancy calendar.

4.3.1 Main Panel: My Warehouses

The central view lists all published warehouses filtered by verification status: Approved, Pending, Occupied, Rejected, and Inactive.

Figure 4.15: My Warehouses: main panel with five warehouses across different verification states.

Figure 4.16: My Warehouses: “Approved” filter showing an active warehouse with one ongoing booking.

4.3.2 Publish Warehouse

A guided seven-step flow for listing a new warehouse: space type, storage modality, photos, location, title and description, pricing and size, and security documentation.

Figure 4.17: Publish Warehouse, Step 1/7: space type selection.

Figure 4.18: Publish Warehouse, Step 2/7: storage modality (complete, private, or shared).

Figure 4.19: Publish Warehouse, Step 3/7: photo upload (minimum 1, recommended 5).

Figure 4.20: Publish Warehouse, Step 4/7: location on map with address and district fields.

Figure 4.21: Publish Warehouse, Step 5/7: title and description of the space.

Figure 4.22: Publish Warehouse, Step 6/7: monthly price and surface area (m²) configuration.

Figure 4.23: Publish Warehouse, Step 7/7: security features, cancellation policy, and mandatory fire department permit.

4.3.3 Incoming Bookings

A table view of all bookings across the manager’s warehouses with global metrics and filters by client, payment status, and date range.

Figure 4.24: Incoming Bookings: table with six bookings, key metrics, and payment status indicators.

4.3.4 Messages

The manager’s inbox organises conversations by client and warehouse, distinguishing between pre-booking inquiries and active rental chats.

Figure 4.25: Manager Messages: client conversations organised by warehouse and booking status.

4.3.5 Calendar

Monthly calendar view displaying all bookings and pending payments, colour-coded by status.

Figure 4.26: Calendar: monthly view (June 2026) with active bookings and pending payment events.

4.3.6 Profile Settings

Profile screen for updating personal information and switching the panel view between available system roles.

Figure 4.27: Profile Settings: personal data editor with role view selector.

4.4 Administrator

The Administrator has full platform control: warehouse listing approval, user account management, incident report tracking, and direct messaging with participants.

4.4.1 Dashboard

Platform-wide metrics: active warehouses, pending approvals, monthly bookings, and open reports, plus a pending listings queue and recent activity log.

Figure 4.28: Admin Dashboard: platform overview showing 14 active warehouses, 5 pending approvals, 32 bookings, and 3 open reports.

4.4.2 Warehouse Approval

The approval queue presents submitted listings with photo galleries, property details, and the fire department permit.

Figure 4.29: Warehouse Approval: review queue with photos, details, permit verification, and approve/reject actions.

Figure 4.30: Reject Warehouse Modal: rejection reason selection with optional comment sent to the manager.

4.4.3 Account Management

Lists all registered accounts with role, registration date, warehouse count, and status. The admin can block or reactivate accounts.

Figure 4.31: Account Management: user table with role, registration date, warehouse count, and account status.

4.4.4 Reports

Tracks platform incidents by type (Access, Security, Infrastructure) and resolution status.

Figure 4.32: Reports: incident table with user, address, date, type, and resolution status.

4.4.5 Messages

Direct communication with platform users for support and moderation.

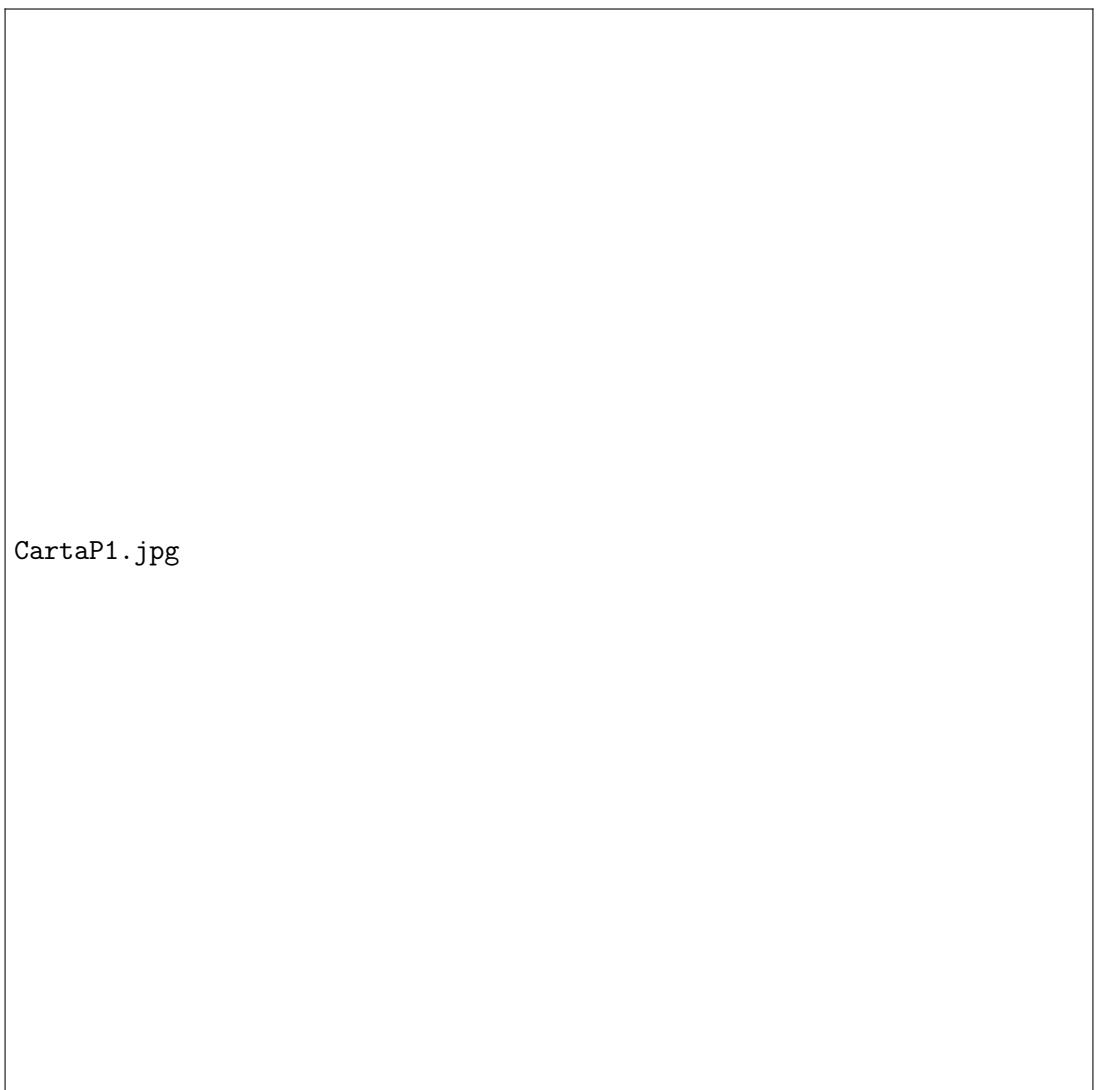
Figure 4.33: Admin Messages: inbox with user conversations for platform support and moderation.

4.4.6 Profile Settings

Figure 4.34: Admin Profile Settings: personal data editor with role view selector.

Chapter 5

Acceptance Letter



CartaP1.jpg

CartaP2.jpg

Chapter 6

Links

- **Repositorio (Excel with user histories):** https://espolec-my.sharepoint.com/:x:/g/personal/dldiaz_esp1_edu_ec/IQBBQnkoG3uFRa2nX5pI2kURAA8pNF6cpxQ1P0tHXTjjAUw?e=1B57H0
- **Prototipo:** <https://darwin4050e.github.io/PrototipoLeodega/>
- **Project files (Folder with meet recording):** https://espolec-my.sharepoint.com/:f:/g/personal/dldiaz_esp1_edu_ec/IgCEzk7GtyQvTI93FPW-tZN4Aaz2tQWK37bkN083-wlybdM?e=1sSQTk

Chapter 7

Product Owner Meeting Records

Throughout the project, communication with the stakeholder took place through two main channels: Microsoft Teams and WhatsApp. This combination was necessary because the team members had conflicting schedules that made it difficult to coordinate frequent virtual meetings. Whenever availability allowed, the team held formal sessions over Microsoft Teams; otherwise, WhatsApp was used as a quicker alternative for brief questions or for staying in touch with the stakeholder between scheduled meetings.

Table 7.1: Meetings

Channel	Date	Attendance	Time	Resume
Microsoft Teams	May 21th	Complete	27min	Leonardo introduced Leodega as a warehouse or inventory system for managing and renting storage spaces. He also noted API integrations, UI-based field mapping, and that the project is already in progress.
WhatsApp	May 22th	–	–	The client shared the deployment manuals from the previous project (Laravel and Frontend) as reference.
WhatsApp	May 24th	–	–	Request for administrator credentials to review the existing module and define what to improve or add.
WhatsApp	May 26th	–	–	Questions about the booking flow, how long a warehouse can stay in the system, and the business model.
Microsoft Teams	May 29th	Complete	25min	Review of the project’s user stories (Warehouse Manager, Client, Administrator) in the shared document.
WhatsApp	June 4th	–	–	Explanation of the multi-company scheme, the Leodega web platform and the “Leodeguita” mobile app share the same warehouse model and backend.
WhatsApp	June 7th	–	–	Resolution of the team’s final doubts about the integration between Leodega and Leodeguita.
Microsoft Teams	June 22th	Complete	40min	Review of the Leodega functional prototype (dashboard, messages, reservations) demonstrated live.

Appendix A

User Stories (Functional Requirements)

This appendix details the functional requirements of the Leodega platform as User Stories, following the format prescribed in `document_structure.md`. Each story lists its identifier, priority (MoSCoW: *Must*, *Should*, *Could*), estimation in story points, and the sprint in which it is planned; its statement in the canonical “As a . . . , I want . . . , so that . . .” form; and its acceptance criteria expressed as numbered scenarios of the form Context, Event, and Expected result. The requirements are grouped by the five roles that interact with the system: *Storage Manager* (HUG), *Client* (HUC), *Administrator* (HUA), *Organisation / Company* (HUE), and the *Leodeguita* mobile companion (HUL). The user stories are the primary source of functional requirements of this specification.

A.1 Storage Manager (HUG)

A.1.1 HUG-01 — Pre-booking inquiry reception and reply

Priority: Could Have

Estimate: 5 points

Sprint: 4

User story statement. As a Storage Manager, I need to receive and reply to pre-booking inquiry messages from clients interested in my spaces, in order to provide timely information and build trust before the client makes a formal reservation.

Acceptance criteria.

1. **Scenario 1: New inquiry message received.** *Context:* a client has sent an inquiry message about one of the Storage Manager’s spaces. *Event:* the system notifies the Manager about the new message. *Expected result:* the system displays the notification in the Manager’s panel with the client’s name, the inquired space, and the message content, and the Manager can access the conversation thread from the notification.
2. **Scenario 2: Successful reply to the client’s message.** *Context:* the Manager accesses the message thread of an interested client and writes a reply. *Event:* the Manager sends the reply message. *Expected result:* the system registers and delivers the reply to the client, updates the conversation thread for both parties, and marks the inquiry as answered in

the Manager's panel.

3. **Scenario 3: Unanswered inquiry after the time limit.** *Context:* the Manager has one or more pending inquiries that have not been answered yet. *Event:* the system detects that the time limit has elapsed without a reply. *Expected result:* the system displays the unanswered inquiries highlighted as “pending”, sorted from oldest to newest, so the Manager can prioritise replies. The prototype does not define an automatic deadline or penalty.

A.1.2 HUG-02 — Payment-confirmation notification and contract activation

Priority: Should Have

Estimate: 3 points

Sprint: 2

User story statement. As a Storage Manager, I need to receive a notification when a client confirms payment for a reservation, in order to know in real time which contracts have been formalised and are ready to begin the rental period.

Acceptance criteria.

1. **Scenario 1: Successful confirmed payment notification.** *Context:* a client has completed the payment process for a reservation on one of the Manager's spaces. *Event:* the system registers the client's payment confirmation. *Expected result:* the system sends a notification to the Manager with the client name, reserved space, amount paid, and contract dates, and the contract becomes active in the Manager's panel.
2. **Scenario 2: Payment not received after the deadline.** *Context:* the client started a reservation on one of the Manager's spaces but did not complete the payment within the hold time. *Event:* the system detects that the payment time limit has expired without confirmation. *Expected result:* the system does not finalise the reservation, returns the space to the available state, and notifies the Manager that the space is free again.
3. **Scenario 3: Viewing the active contract in the panel.** *Context:* the Manager accesses the contracts panel after receiving the confirmed payment notification. *Event:* the Manager opens the active contracts section. *Expected result:* the system displays the contract with all its details (client, space, dates, amount, and “Active” status), and the Manager can access the client's chat from the same panel.

A.1.3 HUG-03 — Active-contract chat with clients

Priority: Could Have

Estimate: 5 points

Sprint: 4

User story statement. As a Storage Manager, I need to have an active chat channel with my clients during the current rental period, in order to coordinate access, resolve incidents, and maintain direct communication without leaving the platform.

Acceptance criteria.

1. **Scenario 1: Sending a message to a client with an active contract.** *Context:* the Manager has an active contract with a client and wants to send them a message from the contracts panel. *Event:* the Manager writes and sends the message in the contract chat. *Expected result:* the system registers the message, delivers it to the client, and updates the chat thread with the send time, and the client receives a push or on-screen notification.
2. **Scenario 2: Receiving a message from the client with notification.** *Context:* the client has sent a message to the Manager through the chat of an active contract. *Event:* the client's message reaches the system. *Expected result:* the system notifies the Manager with an alert indicating the client's name and the associated contract, and the Manager can reply directly from the notification or from the contracts panel.
3. **Scenario 3: Attempt to chat in a finished contract.** *Context:* the Manager tries to send a message to a client whose rental contract has already ended. *Event:* the Manager tries to write in that contract's chat. *Expected result:* the system disables the text field and displays: "This chat is not available because the rental period has ended." The history remains visible in read-only mode.

A.1.4 HUG-04 — Warehouse registration with fire department permit

Priority: Must Have

Estimate: 5 points

Sprint: 1

User story statement. As a Storage Manager, I need to register a new warehouse on the platform, including dimensions, location, photos, monthly rate, and the valid fire department permit, in order to offer my space available for rent.

Acceptance criteria.

1. **Scenario 1: Successful warehouse registration without additional permits upload.** *Context:* the Manager accesses the warehouse registration form and has all the required space information. *Event:* the Manager completes all required fields, attaches the valid fire department permit, and submits the form. *Expected result:* the system validates the data, stores the permit alongside the warehouse, creates it with "pending verification" status, sends it to the Administrator for review, and shows a success message.
2. **Scenario 2: Registration with permits upload.** *Context:* the Manager is completing the registration form for a new warehouse. *Event:* the Manager completes the required fields but does *not* attach the fire department permit and submits the form. *Expected result:* the system blocks the submission, does not create the warehouse, and displays the message: "You must attach the valid fire department permit to continue."
3. **Scenario 3: Attempt to register with missing required fields.** *Context:* the Manager leaves one or more required fields incomplete. *Event:* the Manager omits a required field. *Expected result:* the system displays an error message indicating the required fields and does not allow submitting the form.

A.1.5 HUG-05 — Reservation list viewing, filtering, and detail

Priority: Must Have

Estimate: 3 points

Sprint: 2

User story statement. As a Storage Manager, I need to view all reservations made for my warehouses, with details of dates, client, and payment status, in order to manage the use of my space and plan logistics.

Acceptance criteria.

1. **Scenario 1: Viewing the reservation list.** *Context:* the Manager has at least one published warehouse with registered reservations. *Event:* the Manager accesses the reservations module on the web page. *Expected result:* the system displays a table with all reservations for their warehouses, including client, warehouse, start and end date, status (paid or canceled), and amount.
2. **Scenario 2: Filtering and searching reservations.** *Context:* the Manager is viewing the reservation list and wants to narrow down the results. *Event:* the Manager uses filters by date range, reservation status, or client name. *Expected result:* the system updates the list showing only the reservations that match the selected criteria.
3. **Scenario 3: Viewing a reservation's detail.** *Context:* the Manager is viewing the reservation list and wants to see the detail of a particular reservation. *Event:* the Manager clicks on a specific reservation. *Expected result:* the system opens a detailed view with the complete client information and the payment receipt.

A.1.6 HUG-06 — Cancellation of paid future reservations

Priority: Must Have

Estimate: 3 points

Sprint: 2

User story statement. As a Storage Manager, I need to cancel a previously paid reservation, indicating a mandatory reason, in order to free up my space in case of unforeseen events and maintain transparency with the client.

Acceptance criteria.

1. **Scenario 1: Successful cancellation of a future reservation.** *Context:* the Manager has a paid reservation with a future start date on one of their warehouses. *Event:* the Manager selects a paid reservation with a future start date, enters a valid reason, and confirms the cancellation. *Expected result:* the system changes the reservation status to “canceled by manager”, notifies the client via email, and frees the space for new reservations.
2. **Scenario 2: Cancellation of an already started or finished reservation.** *Context:* the Manager has a paid reservation whose start date has already passed or that has already finished. *Event:* the Manager tries to cancel that reservation. *Expected result:* the system displays an error message indicating that it is not possible to cancel ongoing or finished reservations.

3. **Scenario 3: Cancellation without a reason.** *Context:* the Manager selects a paid, future reservation and tries to cancel it. *Event:* the Manager tries to cancel without filling in the mandatory reason field. *Expected result:* the system does not allow the cancellation and highlights the reason field as required.

A.1.7 HUG-07 — Warehouse deletion (no active or future reservations)

Priority: Must Have

Estimate: 3 points

Sprint: 1

User story statement. As a Storage Manager, I need to delete a warehouse from the platform, provided it has no active or future reservations, in order to remove spaces that I no longer offer for rent.

Acceptance criteria.

1. **Scenario 1: Successful deletion of a warehouse without reservations.** *Context:* the Manager selects a warehouse they own that has no active or confirmed future reservations. *Event:* the Manager selects “Delete warehouse” and confirms the action in the confirmation dialog. *Expected result:* the system removes the warehouse from the public catalog and the Manager’s listings, marks it as inactive while preserving its history, and displays “The warehouse was deleted successfully.”
2. **Scenario 2: Attempt to delete a warehouse with active or future reservations.** *Context:* the warehouse has at least one ongoing reservation or a confirmed/paid future reservation. *Event:* the Manager tries to delete it. *Expected result:* the system blocks the deletion and displays “You cannot delete this warehouse because it has active or future reservations. Wait for them to end or manage them before deleting it”, without making any changes.
3. **Scenario 3: Deletion of a warehouse with only finished (historical) reservations.** *Context:* the warehouse only has already finished reservations, with none active or future. *Event:* the Manager confirms the deletion. *Expected result:* the system allows the operation but preserves the history of finished reservations for audit and reporting purposes; the warehouse is marked as inactive instead of being physically deleted.

A.1.8 HUG-08 — Warehouse information editing

Priority: Must Have

Estimate: 5 points

Sprint: 1

User story statement. As a Storage Manager, I need to edit the information of a warehouse I have already published, such as dimensions, description, photos, monthly rate, and availability, in order to keep my space’s offer up to date without having to delete and recreate it.

Acceptance criteria.

1. **Scenario 1: Successful warehouse data edit.** *Context:* the Manager accesses a warehouse they own and modifies one or more of its editable fields. *Event:* the Manager saves the changes with all valid data. *Expected result:* the system validates the data, updates the warehouse information, reflects the changes in the catalog and the Manager’s listings, and displays “Changes were saved successfully.”
2. **Scenario 2: Attempt to save with missing or invalid required fields.** *Context:* the Manager clears a required field or enters an invalid value (for example, a zero or negative rate). *Event:* the Manager tries to save the changes. *Expected result:* the system highlights the fields with errors, displays a validation message describing what to fix, and does not save any changes.
3. **Scenario 3: Editing a warehouse with active or future reservations.** *Context:* the Manager modifies the rate or other data of a warehouse that already has confirmed or ongoing reservations. *Event:* the Manager saves the changes. *Expected result:* the system applies the changes only to new reservations; already confirmed reservations keep the conditions (rate and dates) they were agreed upon with, and the system informs the Manager that the changes do not affect existing contracts.

A.2 Client (HUC)

A.2.1 HUC-01 — Warehouse search, filtering, and map view

Priority: Must Have

Estimate: 5 points

Sprint: 2

User story statement. As a client, I need to search and filter available warehouses by location, size, and price, in order to find the storage space that best fits my needs and budget without having to review all options manually.

Acceptance criteria.

1. **Scenario 1: Search with available results.** *Context:* the client enters the warehouse catalog and applies location, minimum size, and price range filters. *Event:* the client presses the search button with the selected filters. *Expected result:* the system displays a list of approved warehouses that meet all the entered criteria, with their name, monthly price, size, and estimated distance, and each result includes a button to view the detail.
2. **Scenario 2: Search with no results for the applied filters.** *Context:* the client applies filters whose combination does not match any available warehouse in the system. *Event:* the client runs the search with those criteria. *Expected result:* the system displays “We didn’t find available warehouses with those criteria. Try broadening your search” without showing empty results or errors, and offers the option to clear the filters.
3. **Scenario 3: Warehouse map view.** *Context:* the client activates the map view from the warehouse catalog. *Event:* the system loads the interactive map. *Expected result:* the system displays georeferenced markers for the available warehouses on the map, and when

clicking a marker a card unfolds with the name, monthly price, and a direct link to that warehouse's detail.

A.2.2 HUC-02 — Warehouse detail view

Priority: Must Have

Estimate: 3 points

Sprint: 2

User story statement. As a client, I need to view the complete detail of a warehouse before making a reservation decision, in order to evaluate whether the space meets my requirements regarding conditions, capacity, price, and location before committing to a reservation.

Acceptance criteria.

1. **Scenario 1: Successful warehouse detail load.** *Context:* the client selects a warehouse from the catalog or from the map. *Event:* the system loads the detail page for that warehouse. *Expected result:* the system displays the warehouse name, image gallery, description, dimensions (m²), monthly price, exact address with embedded map, owning manager's name, and availability, and the "Reserve" button is shown only if the warehouse has available status.
2. **Scenario 2: Warehouse not available for reservation.** *Context:* the client accesses the detail of a warehouse whose current status is "occupied" or "unavailable". *Event:* the system loads the detail of that warehouse. *Expected result:* the system displays all the warehouse information but replaces the "Reserve" button with a label indicating "Currently unavailable", without allowing the reservation flow to start.
3. **Scenario 3: Attempt to access a nonexistent warehouse.** *Context:* the client accesses a warehouse detail URL with an identifier that does not exist or was removed from the system. *Event:* the system tries to load the warehouse with that identifier. *Expected result:* the system displays an error page with "This warehouse does not exist or is no longer available", with a link redirecting to the main catalog.

A.2.3 HUC-03 — Warehouse reservation with date-conflict detection

Priority: Must Have

Estimate: 5 points

Sprint: 2

User story statement. As a client, I need to be able to reserve an available warehouse by selecting the desired rental period, in order to secure the storage space I need for the required dates and receive formal confirmation of the agreement.

Acceptance criteria.

1. **Scenario 1: Reservation successfully created.** *Context:* the client is authenticated, views an available warehouse, and enters valid start and end dates within the availability calendar. *Event:* the client confirms the reservation by pressing the "Confirm reservation" button. *Expected result:* the system creates the reservation in "pending payment" status,

temporarily holds the warehouse’s availability for that period, and redirects the client to the payment flow, showing the reservation summary (warehouse, period, and total amount) with its identifying number.

2. **Scenario 2: Reservation attempt with overlapping dates.** *Context:* the client selects a date range that overlaps with an already confirmed reservation for that same warehouse. *Event:* the client tries to confirm the reservation with those dates. *Expected result:* the system blocks the operation, displays “The selected dates are not available. Choose another period”, and visually highlights the occupied periods in the calendar without creating any record.
3. **Scenario 3: Reservation attempt without an active session.** *Context:* an unauthenticated user presses the “Reserve” button on a warehouse detail page. *Event:* the system detects that there is no active session when trying to start the reservation flow. *Expected result:* the system redirects the user to the login form, displaying “You must log in to make a reservation”, and once authenticated it automatically redirects to the same warehouse.

A.2.4 HUC-04 — Pre-booking message to manager

Priority: Could Have

Estimate: 3 points

Sprint: 4

User story statement. As a client, I need to be able to send a message to a warehouse’s manager before making a reservation to clarify questions about the space, in order to obtain information that is not in the description and make a more informed reservation decision without committing beforehand.

Acceptance criteria.

1. **Scenario 1: Message successfully sent to the manager.** *Context:* the authenticated client is viewing a warehouse detail and has a specific question about the space. *Event:* the client writes a message in the inquiry form and presses “Send message”. *Expected result:* the system registers the message linked to the warehouse and the client, notifies the manager via an internal alert, and shows the client the confirmation: “Your message was sent. The manager will reply shortly.”
2. **Scenario 2: Client receives a reply from the manager.** *Context:* the manager has replied to the pre-booking inquiry the client sent about their warehouse. *Event:* the client accesses the “My inquiries” module or enters the inquired warehouse. *Expected result:* the system displays the conversation thread with messages in chronological order, identifying each participant, and the client can continue the conversation by sending an additional message from the same thread.
3. **Scenario 3: Inquiry attempt without an active session.** *Context:* an unauthenticated user tries to send an inquiry message to the manager from a warehouse detail page. *Event:* the user presses the “Ask the manager” button without being logged in. *Expected result:* the system redirects to the login form with “You must log in to contact the manager”, and once authenticated it automatically redirects to the warehouse detail with the inquiry

form visible.

A.2.5 HUC-05 — Simulated payment confirmation

Priority: Must Have

Estimate: 5 points

Sprint: 2

User story statement. As a client, I need to complete a simulated payment process to officially confirm and activate my reservation, in order to formalise the rental agreement and activate the reservation so that both parties receive official confirmation of the contract.

Acceptance criteria.

1. **Scenario 1: Simulated payment successfully completed.** *Context:* the client has a reservation with “pending payment” status and accesses the payment flow. *Event:* the client reviews the payment summary (warehouse, period, total amount) and presses the “Confirm and pay” button. *Expected result:* the system registers the simulated payment, changes the reservation status to “confirmed and active”, notifies the manager that the payment was completed, and shows the client a receipt with the reservation number, warehouse, dates, amount, and “PAID” status.
2. **Scenario 2: Payment attempt after the deadline.** *Context:* the client started a reservation but left the payment flow without completing it within the space’s hold time. *Event:* the client presses the payment button for that reservation. *Expected result:* the system does not finalise the reservation, releases the availability of the warehouse that was held, and displays “You did not complete the payment, so the reservation was not finalized. You can start a new reservation whenever you want.”
3. **Scenario 3: Viewing the payment receipt.** *Context:* the client wants to review the receipt for a reservation they already paid for and that is active. *Event:* the client accesses the detail of that reservation from their “My reservations” panel. *Expected result:* the system displays the full receipt with the reservation number, warehouse name, manager name, start and end dates, total amount paid, payment date and time, and “CONFIRMED” status, and the client can download the receipt as a PDF.

A.2.6 HUC-06 — Active-reservation chat

Priority: Could Have

Estimate: 5 points

Sprint: 4

User story statement. As a client, I need to be able to communicate with the manager through a chat linked to my active reservation during the rental period, in order to coordinate access, report minor issues, and resolve operational situations without needing to involve the administrator in every interaction.

Acceptance criteria.

1. **Scenario 1: Sending a message in an active reservation chat.** *Context:* the client has a reservation with “confirmed and active” status and needs to communicate with the manager. *Event:* the client accesses the chat from their active reservation detail, writes a message, and presses “Send”. *Expected result:* the system registers the message linked to the reservation, displays it in the conversation thread with the timestamp, and notifies the manager via an internal alert; the message is visible to both parties in chronological order.
2. **Scenario 2: Attempt to use the chat in a non-active reservation.** *Context:* the client tries to access the chat of a reservation that has already ended, was canceled, or is still pending confirmation. *Event:* the client navigates to the detail of that reservation and tries to open the chat. *Expected result:* the system disables the writing field and displays “The chat is only available during the active period of the reservation”, and the previous message history remains visible in read-only mode.
3. **Scenario 3: Receiving a message from the manager in the chat.** *Context:* the manager has sent a message to the client within the chat thread of an active reservation. *Event:* the client accesses the platform or has an active session. *Expected result:* the system displays a new message notification in the client’s panel, and when accessing the corresponding reservation’s chat the manager’s message appears in the thread with their name, content, and timestamp, and it is automatically marked as read.

A.2.7 HUC-07 — Reservation history management and cancellation

Priority: Should Have

Estimate: 3 points

Sprint: 4

User story statement. As a client, I need to view and manage my active and historical reservations from my personal panel, in order to have visibility and control over my current rental contracts and be able to cancel a reservation when I no longer need it.

Acceptance criteria.

1. **Scenario 1: Viewing the reservation history.** *Context:* the authenticated client accesses the “My reservations” module from their personal panel. *Event:* the system loads the client’s reservations screen. *Expected result:* the system displays a table with all of the client’s reservations, grouped by status (active, pending, finished, and canceled), with the warehouse name, start and end dates, total price, and current status of each reservation.
2. **Scenario 2: Canceling an active reservation.** *Context:* the client has a reservation with “confirmed” status and wants to cancel it before the start date. *Event:* the client selects the “Cancel reservation” option and confirms the action in the confirmation dialog. *Expected result:* the system changes the reservation status to “canceled”, releases the warehouse’s availability for that period, notifies the manager of the cancellation, and immediately reflects the status change in the client’s panel.
3. **Scenario 3: Attempt to cancel an already started reservation.** *Context:* the client tries to cancel a reservation whose start date has already passed and that is in “ongoing” status. *Event:* the client presses the cancel button on that reservation. *Expected result:* the

system blocks the cancellation, hides or disables the cancel button for ongoing reservations, and displays “This reservation is already ongoing and cannot be canceled from here. Contact the administrator if you have a problem.”

A.2.8 HUC-08 — Incident reporting and tracking

Priority: Could Have

Estimate: 3 points

Sprint: 4

User story statement. As a client, I need to be able to report a problem related to a warehouse or an active reservation from my personal panel, in order to notify the administrator about breaches, damages, or anomalous situations and receive follow-up on my case without relying on external channels.

Acceptance criteria.

1. **Scenario 1: Report successfully created.** *Context:* the client has an active or finished reservation and wants to report an incident related to that warehouse. *Event:* the client completes the report form (selecting the problem type and a mandatory description of at least 20 characters) and presses “Send report”. *Expected result:* the system registers the report with “pending” status, links the report to the reservation and the corresponding warehouse, notifies the administrator, and shows the client the generated case number with “Your report was sent. We will notify you about its resolution.”
2. **Scenario 2: Tracking the status of a sent report.** *Context:* the client has previously sent a report and wants to know its current status. *Event:* the client accesses the “My reports” module from their personal panel. *Expected result:* the system displays the client’s report list with the case number, creation date, involved warehouse, current status (pending, in review, or resolved), and, if the administrator sent a message, a new message indicator next to the corresponding report.
3. **Scenario 3: Attempt to send a report without a description.** *Context:* the client tries to send a report without completing the mandatory description field or with fewer than 20 characters. *Event:* the client presses the “Send report” button with the description field empty or insufficient. *Expected result:* the system blocks the submission, highlights the description field with a red border, and displays “The problem description is mandatory and must be at least 20 characters long”, without creating any record in the database.

A.2.9 HUC-09 — Warehouse and manager rating and review

Priority: Could Have

Estimate: 3 points

Sprint: 4

User story statement. As a client, I need to be able to rate and leave a review about the manager and the warehouse once my rental period has ended, in order to share my experience

with other platform users and contribute to the managers' reputation, helping future clients make better decisions.

Acceptance criteria.

1. **Scenario 1: Review successfully submitted.** *Context:* the client has a reservation with “finished” status and has not yet left a review for that warehouse. *Event:* the client selects a score from 1 to 5, writes a comment of at least 10 characters, and presses “Submit review”. *Expected result:* the system registers the review linked to the warehouse and the manager, displays it publicly on the warehouse profile, updates the manager's average rating, and notifies the manager that they received a new review.
2. **Scenario 2: Review attempt on an unfinished reservation.** *Context:* the client tries to leave a review about a warehouse whose reservation is still active or has not reached its end date. *Event:* the client looks for the rating option on that reservation from their personal panel. *Expected result:* the system does not show the rating option for reservations that do not have “finished” status; if the client accesses directly by URL, the system displays “You can only rate a warehouse once your rental period has ended.”
3. **Scenario 3: Attempt to leave a second review on the same reservation.** *Context:* a client who already left a review for a finished reservation tries to submit a second review for that same reservation. *Event:* the client accesses the detail of that finished reservation and tries to rate again. *Expected result:* the system detects that a review from that client already exists for that reservation, hides the rating form, and shows the review they already submitted with the option to view it, but not to modify it or add a new one.

A.2.10 HUC-10 — External inventory endpoint registration

Priority: Should Have

Estimate: 8 points

Sprint: 5

User story statement. As a client, I need to register the endpoint URL of my external inventory system along with the access credential that said system provided me, in order for Leodega to automatically query the products stored in my warehouse, without me having to modify my system or load the products manually.

Acceptance criteria.

1. **Scenario 1: Successful connection configuration.** *Context:* the client has at least one active reservation and accesses the integrations section of their account. *Event:* the client enters the endpoint URL and access credential, and presses “Test and save connection”. *Expected result:* the system performs a test query to the endpoint; if it responds correctly, it saves the configuration encrypted, displays “Connection verified and saved”, and enables inventory synchronisation for that warehouse.
2. **Scenario 2: Invalid endpoint or credential.** *Context:* the URL does not respond, does not exist, or the credential is rejected by the external system. *Event:* the client presses “Test and save connection”. *Expected result:* the system does not save the configuration, displays “We could not connect to the endpoint. Verify the URL and credential”, and

indicates the type of error detected (no response, unauthorised, or invalid format).

3. **Scenario 3: Updating an existing connection.** *Context:* the client changes their external system or the company modifies the URL or credential. *Event:* the client edits the connection data and saves again. *Expected result:* the system replaces the previous configuration, re-validates the connection, and applies the new data as of the next inventory query.

A.2.11 HUC-11 — External inventory field mapping

Priority: Should Have

Estimate: 8 points

Sprint: 5

User story statement. As a client, I need to configure how the fields returned by my external system's endpoint correspond to Leodega's fields, in order for my inventory data to be registered correctly, without needing to modify my system or contact a developer.

Acceptance criteria.

1. **Scenario 1: Initial mapping configuration.** *Context:* the client has saved a valid connection and accesses the mapping configuration for the first time. *Event:* the system fetches a sample of the endpoint's response. *Expected result:* the system automatically detects the fields present in the response and displays them in a list on the left, and on the right the client assigns which Leodega field each one corresponds to (product name, quantity, unit of measure, expiration date, etc.).
2. **Scenario 2: Saving and validating the mapping.** *Context:* the client has completed the field assignment and presses save. *Event:* the system tries to save the configuration. *Expected result:* the system validates that Leodega's required fields (product name and quantity) have an assigned field; if any are missing, it indicates which are required; if everything is correct, it saves and confirms with "Mapping saved. Your inventory will sync on the next query."
3. **Scenario 3: Modifying the existing mapping.** *Context:* the external system changes its field names. *Event:* the client updates and saves a new mapping. *Expected result:* the system replaces the previous configuration and applies the new mapping as of the next inventory query.

A.2.12 HUC-12 — Synchronised inventory viewing

Priority: Could Have

Estimate: 5 points

Sprint: 5

User story statement. As a client, I need to see the updated list of products stored in my reserved space on the platform, obtained directly from my external system, in order to control my inventory without having to contact the warehouse manager or rely on external channels.

Acceptance criteria.

1. **Scenario 1: Viewing the inventory queried from the endpoint.** *Context:* the client has an active reservation and a valid connection with their external system configured. *Event:* the client accesses the inventory view of their reserved space. *Expected result:* the system queries the endpoint at that moment, applies the configured mapping, and displays a table with the products, indicating for each one the name, current quantity, unit of measure, and the date and time of the query.
2. **Scenario 2: The endpoint does not respond or the query fails.** *Context:* the external system is down, takes too long to respond, or returns an error at query time. *Event:* the client accesses the inventory view. *Expected result:* the system displays “We could not get your inventory at this moment. We are showing the latest available information” along with the last queried data and its date; if there was never any data, it offers to retry or import an Excel file.
3. **Scenario 3: Space with no connection configured yet.** *Context:* the client has an active reservation but has not configured the connection with their external system. *Event:* the client accesses the inventory section of their space. *Expected result:* the system displays “You have not connected your inventory system yet. Configure the connection or import an Excel file to get started”, with direct access to both options.

A.2.13 HUC-13 — Excel inventory import

Priority: Could Have

Estimate: 5 points

Sprint: 5

User story statement. As a client, I need to be able to upload an Excel file with the list of my products directly to the platform, in order to register my inventory quickly when my external system does not expose an endpoint, without having to enter each product manually or configure a technical integration.

Acceptance criteria.

1. **Scenario 1: Successful import of a valid file.** *Context:* the client has an active reservation and uploads an Excel file with the required columns (product name and quantity as a minimum). *Event:* the client confirms the import. *Expected result:* the system processes the file, registers all products in the corresponding space’s inventory, and displays a summary indicating how many products were imported correctly.
2. **Scenario 2: File with incomplete rows or incorrect format.** *Context:* the uploaded Excel file has rows missing the product name or quantity. *Event:* the system processes the file. *Expected result:* the system imports only the valid rows, omits those with errors, and displays a detailed report indicating which rows were ignored and why, without canceling the entire import.
3. **Scenario 3: Downloading an example template.** *Context:* the client does not know what structure their Excel file should have. *Event:* the client selects the “Download template” option from the import section. *Expected result:* the system downloads an Excel file with the expected columns already defined in the first row and sample data in the following

rows, ready to be replaced by the client.

A.2.14 HUC-14 — Organisation invitation acceptance or rejection

Priority: Must Have

Estimate: 3 points

Sprint: 3

User story statement. As a client, I need to be able to accept or reject an invitation that an organisation has sent me to join as a member, in order to decide whether I want to operate under that organisation's name within the platform.

Acceptance criteria.

1. **Scenario 1: Successful invitation acceptance.** *Context:* the client has a pending invitation from an organisation and accesses their notifications panel. *Event:* the client presses the “Accept” button on the received invitation. *Expected result:* the system links the client as a member of the organisation, updates their profile, notifies the Organisation Administrator, and enables the context selector (personal / company) in the client's interface.
2. **Scenario 2: Invitation rejection.** *Context:* the client receives an invitation but does not want to join that organisation. *Event:* the client presses the “Reject” button on the received invitation. *Expected result:* the system discards the invitation, notifies the Organisation Administrator that it was rejected, and makes no changes to the client's profile or permissions.
3. **Scenario 3: Expired invitation.** *Context:* the client tries to respond to an invitation that has already exceeded the system's validity period. *Event:* the client presses “Accept” or “Reject” on an expired invitation. *Expected result:* the system displays “This invitation is no longer available. Ask the organisation administrator to send you a new one” and automatically archives the invitation.

A.3 Administrator (HUA)

A.3.1 HUA-01 — Fire department permit validation

Priority: Could Have

Estimate: 5 points

Sprint: 1

User story statement. As an administrator, I need to review and validate the valid fire department permit document that the warehouse manager uploaded to the system, in order to ensure compliance with the mandatory legal requirement before enabling any storage space on the platform.

Acceptance criteria.

1. **Scenario 1: Approval with valid document.** *Context:* a manager has submitted a warehouse publication request and has attached a valid fire department permit document.

Event: the administrator reviews the document, verifies its validity, and selects the approve option. *Expected result:* the system changes the warehouse status to “approved”, makes it visible in the public catalog, and sends a notification to the manager informing them that their space was enabled.

2. **Scenario 2: Rejection due to expired or invalid document.** *Context:* the administrator is reviewing the attached permit and determines that it is expired or does not correspond to the declared property. *Event:* the administrator selects the reject option and enters a mandatory reason. *Expected result:* the system keeps the warehouse in “rejected” status, prevents it from being visible in the catalog, and sends a notification to the manager with the rejection reason, indicating that they must upload a valid document to restart the process.
3. **Scenario 3: Request submitted without an attached document.** *Context:* a manager tries to submit their warehouse publication request without having uploaded the fire department permit. *Event:* the manager presses the final submit button in the warehouse registration flow. *Expected result:* the system blocks the submission, does not create the warehouse record, and displays “You must attach the valid fire department permit to continue.”

A.3.2 HUA-02 — Administrative control dashboard

Priority: Could Have

Estimate: 8 points

Sprint: 5

User story statement. As an administrator, I need to view a control dashboard with key metrics about the platform’s status, in order to make informed decisions about the business operation and monitor the monthly profitability generated.

Acceptance criteria.

1. **Scenario 1: Loading the dashboard with general metrics.** *Context:* the administrator enters the dashboard module from the main menu. *Event:* the administrator accesses the dashboard and the system loads the screen. *Expected result:* the system displays summary cards with the total registered warehouses (broken down by status: approved, pending, and rejected), the total active users broken down by role (manager and client), the total transacted amount, and the total reservations in progress at that moment.
2. **Scenario 2: Querying the reservation flow by period.** *Context:* the administrator selects a specific month and year in the dashboard’s time filter. *Event:* the administrator applies the filter. *Expected result:* the system updates the metrics showing the number of confirmed reservations in that period, the total transacted amount, and a bar or line chart comparing that period with the previous month.
3. **Scenario 3: Unresolved reports indicator.** *Context:* there are reports with “pending” status registered in the system. *Event:* the administrator accesses the dashboard. *Expected result:* the system displays a numeric indicator of unresolved reports classified by priority (high, medium, and low), with a direct link to the report management module.

A.3.3 HUA-03 — User account blocking and reactivation

Priority: Must Have

Estimate: 5 points

Sprint: 1

User story statement. As an administrator, I need to be able to block or reactivate registered user accounts on the platform from a dedicated management panel, in order to act as a moderator against violations of usage policies and protect the integrity of the community.

Acceptance criteria.

1. **Scenario 1: Blocking an active account.** *Context:* the administrator has identified a user who violated the platform’s policies and selects their profile in the user management panel. *Event:* the administrator confirms the account blocking action and enters the mandatory blocking reason. *Expected result:* the system changes the user’s status to “blocked”, immediately invalidates all their active access tokens, records the action in the audit log with the entered reason, and notifies the user that their account has been suspended.
2. **Scenario 2: Login attempt by a blocked user.** *Context:* a user with “blocked” status tries to authenticate on the platform with correct credentials. *Event:* the user enters their email and password and presses “Log in”. *Expected result:* the system rejects the access and displays “Your account has been suspended. Contact support for more information”, without revealing whether the credentials are valid or incorrect.
3. **Scenario 3: Reactivating a blocked account.** *Context:* the administrator decides to reactivate the account of a user who was previously blocked and whose situation has been resolved. *Event:* the administrator selects the reactivate account option from the management panel. *Expected result:* the system changes the user’s status to “active”, records the reactivation in the audit log, and allows the user to log in normally on their next attempt.

A.3.4 HUA-04 — Report review and resolution

Priority: Could Have

Estimate: 3 points

Sprint: 4

User story statement. As an administrator, I need to review and follow up on the reports that clients send about warehouses or reservations, being able to reply to them and change their status, in order to handle incidents, maintain service quality, and close the loop on each report.

Acceptance criteria.

1. **Scenario 1: Reviewing a pending report.** *Context:* there is at least one report with “pending” status sent by a client. *Event:* the administrator accesses the report management module and opens one. *Expected result:* the system displays the report detail (type, description, linked warehouse and reservation, client, and date) and allows changing its status to “in review”.

2. **Scenario 2: Replying to and resolving the report.** *Context:* the administrator has reviewed a report and has a response or resolution. *Event:* the administrator writes a message to the client and marks the report as “resolved”. *Expected result:* the system changes the status to “resolved”, records the response, notifies the client with a new message indicator, and decrements the report from the dashboard’s pending counter.
3. **Scenario 3: Attempt to resolve without a reply.** *Context:* the administrator tries to mark a report as “resolved” without having entered a response for the client. *Event:* the administrator confirms the resolution with the response field empty. *Expected result:* the system blocks the action and requests entering a response before closing the report.

A.4 Organisation / Company (HUE)

A.4.1 HUE-01 — Organisation member invitation

Priority: Must Have

Estimate: 5 points

Sprint: 3

User story statement. As an Organisation Administrator, I need to be able to invite users registered as clients to join my organisation as members, in order for my collaborators to be able to operate under the company’s name on the platform.

Acceptance criteria.

1. **Scenario 1: Invitation successfully sent.** *Context:* the Administrator searches for a user by email and the user exists with a client role. *Event:* the administrator selects the user and confirms sending the invitation. *Expected result:* the system registers the invitation with “pending” status and notifies the client via an alert in their panel and an email with the organisation’s name and buttons to accept or reject.
2. **Scenario 2: Attempt to invite a warehouse manager.** *Context:* the Administrator tries to search for and invite a user with a Storage Manager role. *Event:* the administrator enters a manager’s email in the invitation search. *Expected result:* the system blocks the action and displays “You can only invite users with a client role. Warehouse managers cannot join organizations.”
3. **Scenario 3: Attempt to invite a user who is already a member.** *Context:* the client being invited is already an active member of the organisation. *Event:* the administrator selects that user and confirms the invitation. *Expected result:* the system displays “This user is already a member of your organization” and does not generate a duplicate invitation record.

A.4.2 HUE-02 — Personal / organisation context switching

Priority: Must Have

Estimate: 5 points

Sprint: 3

User story statement. As a Client (belonging to an organisation), I need to be able to switch between my personal context and that of each organisation from a selector visible in the web interface, in order to operate under the correct name depending on the situation, without needing to log out or create separate accounts.

Acceptance criteria.

1. **Scenario 1: Switching from personal to organisation context.** *Context:* the client has an active session in personal context and belongs to at least one organisation. *Event:* the client clicks the context selector (circular indicator in the top right) and chooses an organisation's name. *Expected result:* the system switches the active context, updates the visual indicator with the organisation's name/logo, and reloads the panel showing only that organisation's warehouses, reservations, and inventory.
2. **Scenario 2: Switching from organisation to personal context.** *Context:* the client is operating under an organisation's context. *Event:* the client selects "Personal mode" from the context selector. *Expected result:* the system restores the individual context, hides the organisation indicator, and shows the personal panel with their own reservations and activity.
3. **Scenario 3: Login with a single linked organisation.** *Context:* the client belongs to exactly one organisation. *Event:* the client logs in to the platform. *Expected result:* the system displays the context selector pre-selected on "Personal mode", with the option to switch to the organisation visible in the top right corner, and it does not automatically redirect to the business context.

A.4.3 HUE-03 — Business administrative panel

Priority: Should Have

Estimate: 5 points

Sprint: 3

User story statement. As an Organisation Administrator, I need to access a business administrative panel where I can see the members, active reservations, and history of the organisation, in order to have full visibility and control over the operations carried out under my company's name.

Acceptance criteria.

1. **Scenario 1: Loading the business administrative panel.** *Context:* the Organisation Administrator logs in or switches to the business context. *Event:* the administrator accesses the organisation administration panel. *Expected result:* the system displays a summary with the number of active members, current reservations, reservation history, and pending invitations awaiting a response.
2. **Scenario 2: Member management: removing a member.** *Context:* the Administrator wants to remove a client from the organisation. *Event:* the administrator selects a member from the list and confirms their removal. *Expected result:* the system unlinks the client from the organisation, removes their access to the business panel, notifies the client, and preserves the reservation history they made while they were a member.

3. **Scenario 3: Viewing the organisation’s action history.** *Context:* the Administrator wants to audit the actions carried out under the company’s name. *Event:* the administrator accesses the history section of the business panel. *Expected result:* the system displays a chronological record of reservations created/canceled and members added/removed, indicating in each entry the name of the member who performed the action and the date.

A.4.4 HUE-04 — Organisation creation

Priority: Must Have

Estimate: 5 points

Sprint: 3

User story statement. As an Organisation Administrator, I need to be able to create an organisation from my client account, entering the company’s data (business name, RUC, and organisation email), in order to manage warehouse rentals on behalf of my organisation, without having to create a separate account.

Acceptance criteria.

1. **Scenario 1: Successful organisation registration.** *Context:* a client with an active session completes the required organisation data (business name, 13-digit RUC, and organisation email). *Event:* the user presses the “Create organization” button. *Expected result:* the system validates the RUC (13 digits and unique in the system), creates the organisation linked to the client’s account and designates them as the Organisation Administrator (without ceasing to be a client), activates it automatically, generates the business administrative panel, and enables the personal/company context selector.
2. **Scenario 2: Registration attempt with an already existing RUC.** *Context:* the entered RUC is already registered in the system by another organisation. *Event:* the user tries to submit the form with that RUC. *Expected result:* the system blocks the submission, highlights the RUC field, and displays “An organization is already registered with this RUC. If you believe this is an error, contact support.”
3. **Scenario 3: Registration attempt with missing required fields.** *Context:* the user omits one or more required form fields. *Event:* the user tries to submit the incomplete form. *Expected result:* the system highlights each empty required field in red and displays individual validation messages, and no record is created until all required fields are completed.

A.4.5 HUE-05 — Business-level reservation

Priority: Must Have

Estimate: 5 points

Sprint: 3

User story statement. As a Client (belonging to an organisation), I need to be able to reserve an available warehouse on behalf of the organisation I belong to, in order to secure storage space for the company and so that all members can view and manage that reservation.

Acceptance criteria.

1. **Scenario 1: Business reservation successfully created.** *Context:* the client has an organisation's context active, views an available warehouse, and enters valid dates. *Event:* the client confirms the reservation by pressing "Confirm reservation". *Expected result:* the system creates the reservation linked to the organisation (not to the individual client), notifies the warehouse manager, and updates the panel of all active members of that organisation so they see the new reservation.
2. **Scenario 2: Shared view of the reservation by another member.** *Context:* a second member client accesses the business panel after a reservation was made. *Event:* the second member navigates to the reservations section in the organisation's context. *Expected result:* the system displays the reservation with all its details, indicating the name of the member who made it, and both can access the reservation's chat.
3. **Scenario 3: Reservation attempt without an active business context.** *Context:* the member client is operating in their personal context and tries to reserve a warehouse. *Event:* the client confirms the reservation in personal context. *Expected result:* the system processes the reservation as a personal reservation of the client, not as a business one, and the reservation does not appear in the organisation's panel.

A.4.6 HUE-06 — Business reservation deletion control

Priority: Must Have

Estimate: 3 points

Sprint: 3

User story statement. As a Client (belonging to an organisation), I need the system to prevent me from deleting warehouses reserved under the organisation, since that action is reserved for the Organisation Administrator, in order to protect business contracts from accidental or unauthorized deletions by members without sufficient permissions.

Acceptance criteria.

1. **Scenario 1: Delete button not available to a member.** *Context:* a member client accesses the detail of an active organisation reservation. *Event:* the member views the available options in that reservation's detail view. *Expected result:* the system does not show the "Delete reservation" button for the member client, and only the options to view details and access the reservation's chat are shown.
2. **Scenario 2: Organisation administrator can delete a reservation.** *Context:* the Organisation Administrator accesses the detail of an active business reservation. *Event:* the administrator selects "Delete reservation", enters a mandatory reason, and confirms the action. *Expected result:* the system cancels the reservation, releases the warehouse, notifies the manager and all organisation members about the cancellation, and records the reason in the company's history.
3. **Scenario 3: Deletion attempt by a member via direct URL.** *Context:* a member client tries to access the delete action by manipulating the URL. *Event:* the system receives the deletion request with the member client's token. *Expected result:* the system rejects

the operation with a 403 error, makes no changes to the database, and records the failed attempt in the audit log.

A.5 Leodeguita mobile companion (HUL)

A.5.1 HUL-01 — Leodeguita login with Leodega credentials

Priority: Must Have

Estimate: 3 points

Sprint: 1

User story statement. As a user (administrator, warehouse manager, or client), I need to log in to Leodeguita with my Leodega credentials, in order to access my data and functionalities without needing to create a separate account.

Acceptance criteria.

1. **Scenario 1: Successful login.** *Context:* the user has an active account in Leodega. *Event:* they enter their email and password correctly and press “Log in”. *Expected result:* the system authenticates the user and redirects them to the main screen corresponding to their role.
2. **Scenario 2: Incorrect credentials.** *Context:* the user enters an email or password that does not match any registered account. *Event:* they press “Log in”. *Expected result:* the system displays an error message indicating that the credentials are incorrect and does not allow access.
3. **Scenario 3: Empty required fields.** *Context:* the user leaves the email or password field blank. *Event:* they press “Log in”. *Expected result:* the system highlights the empty fields, displays a message indicating they are required, and does not attempt to authenticate.

A.5.2 HUL-02 — Company selection in Leodeguita

Priority: Must Have

Estimate: 3 points

Sprint: 3

User story statement. As a client, I need to select the company under which I want to operate within Leodeguita, in order to view only the warehouses and items corresponding to that company.

Acceptance criteria.

1. **Scenario 1: Client with multiple companies.** *Context:* the client is associated with more than one company in the system. *Event:* they access the company selector and choose a different one from the active one. *Expected result:* the system updates the entire view to show only the warehouses and items corresponding to the selected company.
2. **Scenario 2: Client with a single company.** *Context:* the client is associated with only one company. *Event:* they log in to Leodeguita. *Expected result:* the system automatically sets that company as active and does not display the company selector.

3. **Scenario 3: Client with no associated companies.** *Context:* the client is not associated with any company in the system. *Event:* they log in to Leodeguita. *Expected result:* the system redirects the user to the view corresponding to the client role.

A.5.3 HUL-03 — Mobile warehouse publication for review

Priority: Must Have

Estimate: 5 points

Sprint: 1

User story statement. As a warehouse manager, I need to register an available warehouse space for rent with the minimum possible steps, in order to offer it to clients quickly from my mobile device.

Acceptance criteria.

1. **Scenario 1: Successful submission for review.** *Context:* the manager has completed all required fields on the form. *Event:* they press “Publish space”. *Expected result:* the system registers the warehouse in the backend, creates it with “pending verification” status, and sends it to the Administrator for review; the warehouse is *not* visible in the catalog until the Administrator approves it, and the system confirms to the manager that their space was sent for review.
2. **Scenario 2: Incomplete required fields.** *Context:* the manager has left one or more required fields incomplete. *Event:* they press “Publish space”. *Expected result:* the system highlights the missing fields, displays a descriptive error message, and does not perform the registration.
3. **Scenario 3: Duplicate warehouse name.** *Context:* a warehouse with the same name already exists, registered by the same manager. *Event:* they press “Publish space”. *Expected result:* the system displays a warning indicating that a warehouse with that name already exists and does not perform the registration.

A.5.4 HUL-04 — Published warehouses list (mobile)

Priority: Must Have

Estimate: 2 points

Sprint: 1

User story statement. As a warehouse manager, I need to view the list of warehouses I have published, in order to monitor my spaces and know the current status of each one.

Acceptance criteria.

1. **Scenario 1: There are published warehouses.** *Context:* the manager has warehouses registered in the system. *Event:* they access the “My warehouses” section. *Expected result:* the system displays a list with the name, location, and status (available/occupied) of each published warehouse.
2. **Scenario 2: No published warehouses.** *Context:* the manager has no warehouses registered in the system. *Event:* they access the “My warehouses” section. *Expected result:*

the system displays a message indicating they have no published warehouses and presents the option to publish a new one.

3. **Scenario 3: Manager applies a search filter.** *Context:* the manager wants to narrow down the results by some criterion, such as location or capacity. *Event:* they select or enter a filter and apply it. *Expected result:* the system updates the list showing only the warehouses that meet the indicated criterion.

A.5.5 HUL-05 — Available spaces browsing (mobile)

Priority: Could Have

Estimate: 5 points

Sprint: 5

User story statement. As a client, I need to see all available warehouse spaces in a single list, in order to find a space that fits my storage needs.

Acceptance criteria.

1. **Scenario 1: There are available spaces.** *Context:* there are warehouses marked as available in the system, regardless of whether they were published from Leodega or Leodeguita. *Event:* the client accesses the space exploration section. *Expected result:* the system displays a unified list with all available spaces, indicating the name, location, and capacity of each one.
2. **Scenario 2: No available spaces.** *Context:* there is no warehouse marked as available in the system at the time of the query. *Event:* the client accesses the space exploration section. *Expected result:* the system displays a message indicating that there are no available spaces at this time.
3. **Scenario 3: Client applies a search filter.** *Context:* the client wants to narrow down the results by some criterion, such as location or capacity. *Event:* they select or enter a filter and apply it. *Expected result:* the system updates the list showing only the available warehouses that meet the indicated criterion.

A.5.6 HUL-06 — Warehouse detail view (mobile)

Priority: Must Have

Estimate: 2 points

Sprint: 2

User story statement. As a client, I need to view the detailed information of an available warehouse space, in order to evaluate whether it meets my needs before renting it.

Acceptance criteria.

1. **Scenario 1: Space with complete information.** *Context:* the selected warehouse has all its fields registered. *Event:* the client selects a warehouse from the list. *Expected result:* the system displays the space's full profile with name, description, location, dimensions, capacity, and contact information for the responsible manager.

2. **Scenario 2: Space with partial information.** *Context:* the selected warehouse has optional fields incomplete. *Event:* the client accesses its detail. *Expected result:* the system displays only the fields that contain information and omits empty fields without generating on-screen errors.
3. **Scenario 3: Space no longer available.** *Context:* the warehouse was marked as occupied while the client was viewing it. *Event:* the client refreshes the screen. *Expected result:* the system displays a notice indicating that this space is no longer available and removes it from the active list.

Appendix B

Non-Functional Requirements

This appendix details the non-functional requirements (NFRs) of the Leodega platform. In accordance with `instructions.md`, the NFRs are grouped following Ian Sommerville’s classification, which distinguishes three top-level families of quality requirements: *Product* requirements (constraining the behaviour of the software itself, e.g., efficiency, reliability, usability, security, portability), *Organisational* requirements (arising from the policies, procedures, and development standards of the team and institution), and *External* requirements (arising from factors outside the system and its development, such as regulatory or legislative obligations). Every requirement is stated in testable terms and is accompanied by quantitative or verifiable validation criteria, in compliance with the editorial instruction that every requirement be testable and that validation criteria be included for every non-functional requirement. The NFRs are sourced from `non_functional_requirements.md` and supplement the functional requirements of Appendix A. They do not, by themselves, add new features; they constrain how the features shall behave.

B.1 Product Requirements

B.1.1 Efficiency

RNF-01 — Performance. The system must respond quickly to user actions so the experience feels fluid.

Validation criteria.

1. Pages load in ≤ 3 seconds under normal usage.
2. Map and storage detail views render in ≤ 5 seconds.
3. No action keeps the user waiting more than 8 seconds without a loading indicator.

RNF-10 — Scalability. The system must keep working well as users and listings grow.

Validation criteria.

1. The database includes indexes on the most searched columns (`city`, `status`, `store_room_id`).
2. Listing and detail pages load in ≤ 3 seconds with 5,000 stored listings.
3. No N+1 queries are detected on the main listing endpoints.

B.1.2 Reliability

RNF-02 — Availability. The platform must be available most of the time for all users.

Validation criteria.

1. The website is accessible $\geq 99\%$ of the time each month.
2. Scheduled maintenance is announced in advance and lasts < 4 hours.
3. The landing page and login remain reachable even during backend updates.

B.1.3 Security

RNF-03 — Authentication. Only registered users with a valid token can access protected features.

Validation criteria.

1. Unauthenticated requests to protected routes return a 401 error.
2. A user cannot access another role's features (e.g., a tenant cannot moderate stores).
3. Logging out invalidates the token immediately.

RNF-04 — Data Protection. User data, especially passwords, must be stored securely.

Validation criteria.

1. Passwords are stored encrypted (bcrypt), never in plain text.
2. Password reset links expire after one use or after a time limit.
3. Uploaded report evidence files are validated by type and size (≤ 10 MB).

B.1.4 Usability

RNF-05 — Usability. The interface must be easy to use for landlords, tenants, and administrators.

Validation criteria.

1. Any main action (reserve, confirm, report) is reachable in ≤ 3 clicks.
2. Error messages are clear and tell the user how to fix the problem.
3. Forms provide visual feedback on required fields and validation errors.

B.1.5 Portability

RNF-06 — Compatibility. The web app must work on common browsers and devices.

Validation criteria.

1. The app works correctly on Chrome, Firefox, Edge, and Safari (latest versions).
2. The layout adapts to mobile (≥ 320 px), tablet, and desktop screens.
3. No core flow (login, search, reservation) is blocked on any supported browser.

RNF-07 — Internationalization. The platform must support multiple languages, starting with Spanish and English.

Validation criteria.

1. Users can switch language from the UI without reloading the page.
2. All visible texts are translated in both Spanish and English.
3. The selected language persists during the user's session.

B.2 Organisational Requirements

B.2.1 Implementation Standards

RNF-08 — Maintainability. The code must be clean and follow quality standards to ease future changes.

Validation criteria.

1. Backend passes PHPStan, Psalm, and Laravel Pint without critical errors.
2. Frontend passes ESLint with 0 errors.
3. TypeScript compiles without type errors (`tsc -noEmit`).

RNF-09 — Testability. Key system features must be covered by automated tests.

Validation criteria.

1. PHPUnit runs all backend feature tests with a 100% pass rate.
2. Cypress runs all frontend end-to-end tests with a 100% pass rate.
3. Critical flows (reservation, report, moderation) have at least one passing test.

B.3 External Requirements

B.3.1 Regulatory

RNF-11 — Auditability. The system must keep traceable records of important user actions.

Validation criteria.

1. Each reservation change generates a notification with sender, receiver, and timestamp.
2. Store moderation records include moderator, status, reason, and date.
3. Active sessions show IP and user agent, and can be revoked by the user.

In total, this appendix specifies eleven non-functional requirements distributed across the three Sommerville families: eight *Product* requirements (RNF-01, RNF-02, RNF-03, RNF-04, RNF-05, RNF-06, RNF-07, and RNF-10), two *Organisational* requirements (RNF-08 and RNF-09), and one *External* requirement (RNF-11). Each requirement is testable through the validation criteria listed above, and each is referenced from the summary table in Section ?? of Chapter 2.

Appendix C

Meeting Evidence

This appendix presents the visual evidence of the virtual meetings held with the Product Owner throughout the project, corresponding to the sessions documented in Chapter 6.

Figure C.1: Product Owner Meeting 1 – May 21

Figure C.2: Product Owner Meeting 2 – May 29

Figure C.3: Product Owner Meeting 3 – June 22